

RTL Design Sherpa

Reed-Solomon UART Harness Genesys 2 FPGA board evidence

October 2026

Table of Contents

1 Document Information....7
1.1 Revision History....7
1.2 Scope....8
1.3 Provenance....9
2 Purpose and Geometry....10
3 What Is In It....11
3.1 RTL blocks....11
3.2 Host scripts....13
3.3 Build and validation flow....15
4 How The Pieces Connect....16
4.1 Register path and datapath....16
4.1.1 Figure 2.1: Reed-Solomon AXIS datapath and register path....16
4.2 Clock and reset....18
5 Harness Address Map....19
5.1 Figure 2.2: UART harness address map....20
5.2 Window summary....21
5.3 rs_loop_apb register offsets....23
5.4 By-name access....25
6 AXI4 Memory-to-Memory Datapath....26
6.1 Figure 2.3: AXI4 datapath....27
6.2 Job chain....29
6.3 Memories....30

6.4 Observer..... 31
6.5 Verdict path.....32
7 The FPGA-Testing Choices, And Why.....33
7.1 The dual-solver loop as mutual oracle.....33
7.2 Determinism makes the riBM and Euclid images cross-oracles.....34
7.3 Injector modes map to real failure mechanisms.....35
7.4 Post-encoder injection protects parity symbols.....36
7.5 UART over a debug core, with a lock/readback safety pair.....37
7.6 Genesys 2 over Nexys A7.....38
7.7 AXIS and AXI4 flavors cover both integration styles.....39
8 Operating It.....40
9 What Is Deliberately Not Board-Tested.....41

List of Figures

Figure 2.1: Reed-Solomon AXIS datapath and register path.....16
Figure 2.2: UART harness address map.....20
Figure 2.3: AXI4 datapath.....27

List of Tables

Table 1: Table 0.1: Revision history.....7

1 Document Information

1.1 Revision History

Table 0.1: Revision history.

Revision	Date	Author	Description
0.1	2026-10-06	RTL Design Sherpa	Initial harness guide for the Reed-Solomon Genesys 2 UART loop.
0.2	2026-10-20	RTL Design Sherpa	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.

1.2 Scope

This book describes the RS(252,236) t=8 UART harness on the Digilent Genesys 2: its contents, connectivity, and the motivations behind the FPGA testing choices. It is the harness guide that pairs with the Reed-Solomon Board Validation Report, which carries the run evidence.

1.3 Provenance

The harness description comes from `docs/UART_HARNESS.md`, the living markdown copy in the harness directory. Build facts come from `stable/MANIFEST.md` and `stable/reports/`. The board used for the runs is Digilent Genesys 2 serial `200300B818A0` (Kintex-7 `xc7k325t-2`), programmed and measured with Vivado 2025.1.

2 Purpose and Geometry

This is the Genesys 2 board harness for the RS(252,236) t=8 Reed-Solomon codec. The same loop subjects both key-equation solvers — riBM and Euclid — to a regenerated LFSR pattern that never saw the injected errors, so a corrected block is proven against a reference rather than against the other decoder. You drive the whole thing from a laptop over a 115200-baud UART, and it reports per-block verdicts, bandwidth meters, and a beat-by-beat comparator when both solvers are present.

The geometry is frozen in `build-loop/rtl/rs_loop_cfg_pkg.sv`: RS(252,236), 4 symbols/beat, shortened so n and k are multiples of the beat. That package is the single source of the geometry, the UART baud, and the build identity, so the clock constant and the register map cannot drift apart.

The harness directory moved from `projects/fpga-systems/NexysA7/reed-solomon/` to `projects/fpga-systems/Genesys2/ecc-ip/reed-solomon/` on 2026-10-05. Set `RS_TARGET=genesys2` to build for the Kintex-7 XC7K325T-2; the default `nexys_a7_100t` keeps the original names and report layout byte-identical. Since 2026-10-07 the same tree also carries a small profile for the A7 (`RS_PROFILE=small`, issue #83): RS(64,56) t=4 at 50 MHz, distinct BUILD_ID “RSLS”. The Genesys 2 image matrix is four bitstreams: `rs_loop_genesys2_{axis_ribm,axis_euclid,axi4_ribm,axi4_euclid}.bit` — two integration flavours times two solvers, one solver per bitstream.

3 What Is In It

3.1 RTL blocks

Every path is relative to `projects/fpga-systems/Genesys2/ecc-ip/reed-solomon/` unless it lives in a shared area.

Module	File	Role
`uart_axil_bridge`	`projects/components/utility-ip/converters/rtl/uart_to_axil4/uart_axil_bridge.sv`	UART 115200 8N1 to AXI4-Lite master.
`bridge_rs_loop_axil`	`rtl/bridges/generated/bridge_rs_loop_axil/bridge_rs_loop_axil.sv`	Generated 1-master x 3-slave AXI4-Lite/APB fabric.
`rs_loop_genesys2_top` / `rs_loop_top`	`build-loop/rtl/rs_loop_genesys2_top.sv` / `build-loop/rtl/rs_loop_top.sv`	Board top: Genesys 2 MMCM wrapper, or Nexys A7 100 MHz direct clock.
`rs_loop_harness`	`build-loop/rtl/rs_loop_harness.sv`	Codec loop, register decode, bandwidth meters, observers, verdict tallies.
`rs_loop_cfg_pkg`	`build-loop/rtl/rs_loop_cfg_pkg.sv`	Single source of geometry: RS(252,236), 4 symbols/beat, shortened so n and k are multiples of the beat.
`rs_loop_regs`	Generated from `build-loop/rtl/rs_loop_regs.rdl` via the shared `apb4_to_peakrdl` shim	Host-visible register block: BUILD_ID RSLP, SCRATCH, PROFILE, TOPOLOGY, INJ_CFG, INJ_SEED, etc.
`axis4_master_injector`	`rtl/amba/shared/axis4_master_injector.sv`	LFSR data source plus expected CRC-32, one packet per block.

Module	File	Role
axis4_slave_pattern_check	rtl/amba/shared/axis4_slave_pattern_check.sv	Regenerates the same LFSR pattern and compares beats per decoder.
rs_encoder_core	projects/components/ecc-ip/reed-solomon/rtl/macro/rs_encoder_core.sv	RS(252,236) encoder.
error_injector	projects/components/utility-ip/misc/rtl/error_injector.sv	Post-encoder corruption: modes 0 none, 1 exact count, 2 burst, 3 rate, 4 clusters, 5 localized, 6 badblock, 7 debug.
rs_decoder_core	projects/components/ecc-ip/reed-solomon/rtl/macro/rs_decoder_core.sv	Decoder with selectable KES_ALGO and erasure path.
rs_erasure_unit	projects/components/ecc-ip/reed-solomon/rtl/fub/rs_erasure_unit.sv	Erasure locator / transform used when ERASURE_SUPPORT=1.
rs_axi4_pipeline	build-loop/rtl/rs_axi4_pipeline.sv	Memory-to-memory job chain for the AXI4 flavor.
axis4_intf_observer	projects/components/utility-ip/misc/rtl/axis4_intf_observer.sv	Four AXIS seam observer on window 0x20000.
axi4_intf_master_observer	projects/components/utility-ip/misc/rtl/axi4_intf_master_observer.sv	Four codec AXI4 master-port observer on window 0x10000.

3.2 Host scripts

The host side is layered so the same programs run in cocotb simulation and on the board.

Script	File	Role
`rs_env`	`bin/rs_env.py`	One place this area learns the repo root and puts shared `projects/fpga-systems/bin` plus `build-loop/host` on `sys.path`.
`run_smoke.py`	`bin/run_smoke.py`	Campaign runner: resolves sequences, opens the UART, and runs them through `SequenceRunner`.
`init`	`bin/seq_init.py`	Proves the link and bitstream: BUILD_ID, SCRATCH round-trip, PROFILE, TOPOLOGY.
`smoke`	`bin/seq_smoke.py`	Bypass, clean run, e = t, e = t + 1, deterministic debug walk.
`sweep`	`bin/seq_sweep.py`	Exact-count table for e = 0 .. 2t + 2.
`random`	`bin/seq_random.py`	Fresh data/error seeds, mixed modes and counts. Also hosts the `clusters`, `localized`, and `badblock` campaign classes.
`soak`	`bin/seq_soak.py`	Million-block random soak, run by run, with replayable seeds.
`erasure`	`bin/seq_erasure.py`	Marked runs at f = t, 2t, 2t + 1.

Script	File	Role
`RsLoopDriver`	`build-loop/host/rs_loop.py`	By-name register access, status, bandwidth meters, observer readout.
`rs_loop_programs`	`build-loop/host/rs_loop_programs.py`	The authored-once programs shared by sim and board: `smoke`, `bypass`, `run`, `sweep`, `verdict`.
`host_rs_loop.py`	`build-loop/host/host_rs_loop.py`	CLI front-end: smoke, bypass, run, sweep, random, bw, obs, soak, erasure.

3.3 Build and validation flow

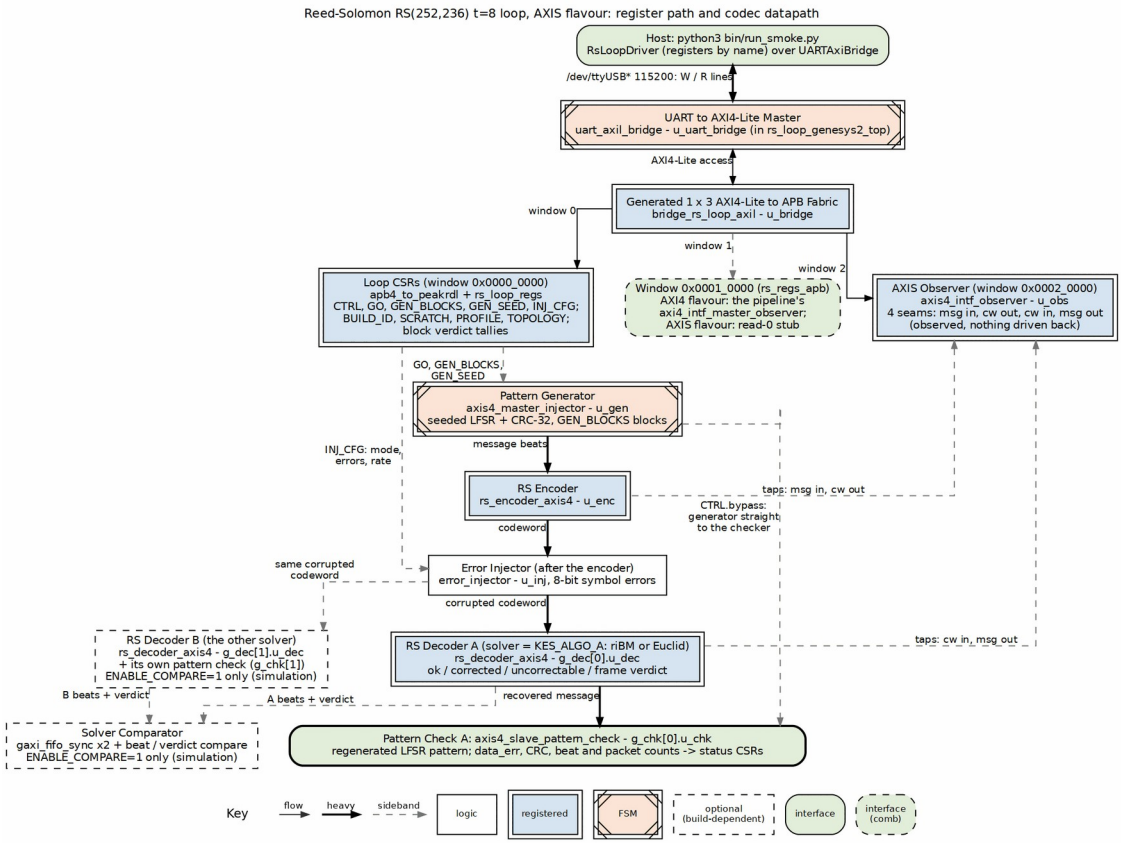
Target / artifact	File	Role
Top-level dispatcher	`Makefile`	Delegates to `build-loop/Makefile` for `bitstream`, `lint`, `sim`, etc.; `make regmap` regenerates CSRs.
Per-build flow	`build-loop/Makefile`	Vivado project, constraints, and report handling; consumes `make/fpga_flow.mk`.
Four-image matrix	`bin/build_image_matrix.sh`	Builds `{axis,axi4}` x `{riBM,Euclid}` with one solver per bitstream.
Bandwidth matrix	`bin/measure_image_matrix.sh`	Programs and measures each image in turn.
Genesys 2 constraints	`build-loop/fpga/constraints/rs_loop_genesys2.xdc`	200 MHz LVDS input, MMCM, UART pins, LEDs.
Validation record	`stable/MANIFEST.md`	Timing slack, board campaigns, and known issues for the kept images.

4 How The Pieces Connect

4.1 Register path and datapath

The host register path is: Python over pyserial to the UART, into `uart_axil_bridge`, through the generated `bridge_rs_loop_axil` 1x3 fabric, and out as three APB windows.

4.1.1 Figure 2.1: Reed-Solomon AXIS datapath and register path



Reed-Solomon AXIS datapath and register path

Source: 01_rs_axis_loop.dot

Window 0 at `0x00000000` is `rs_loop_apb`, which feeds the shared `apb4_to_peakrdl` shim and then the generated `rs_loop_regs` block — that is where BUILD_ID, SCRATCH, PROFILE, TOPOLOGY, INJ_CFG, and the status/counter registers live. Window 1 at `0x00010000` is `rs_regs_apb`; on the AXI4 flavor it hosts the `axi4_intf_master_observer` watching the codec's four AXI4 master ports, while on AXIS flavors it is a read-zero stub so the host bus never hangs. Window 2 at `0x00020000` is `obs_apb`, always live, where an `axis4_intf_observer` watches the four AXIS seams in codeword order: message-in, codeword-out, codeword-in, message-out. The

fabric definition is frozen in `rtl/bridges/generated/bridge_rs_loop_axil/bridge_rs_loop_axil.toml`, so adding a new window is a bridge regeneration rather than a harness rewrite.

4.2 Clock and reset

Clock and reset differ only at the board top. On the Genesys 2 the 200 MHz LVDS system clock passes through `IBUFDS` into an `MMCME2_BASE` with VCO = 1200 MHz and `CLKOUT0_DIVIDE_F = 12`, producing the 100 MHz harness clock (`build-loop/rtl/rs_loop_genesys2_top.sv`). The Nexys A7 top uses the on-board 100 MHz clock directly (`build-loop/rtl/rs_loop_top.sv`). Both synchronize the active-low pushbutton reset into the 100 MHz domain before it reaches `rs_loop_harness`. All of the downstream geometry — including the UART divisor — comes from `rs_loop_cfg_pkg.sv`, so the clock constant and the MMCM output cannot drift apart.

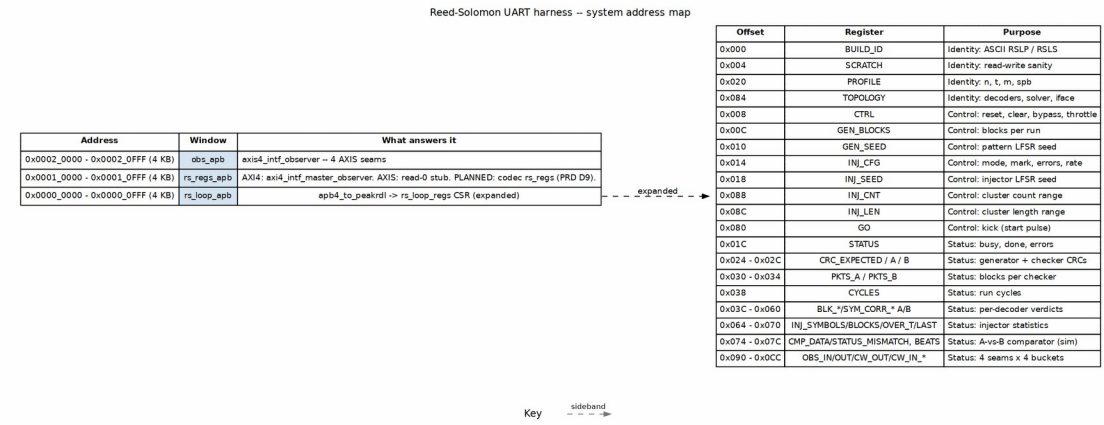
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Harness Address Map

The host sees one flat 32-bit AXI4-Lite address space. The UART bridge (`uart_axil_bridge`) is the master, and the generated `bridge_rs_loop_axil` fabric decodes it into three 4 KB APB windows. Everything the host can reach is in those windows; adding a new window is a bridge regeneration, not a harness rewrite.

5.1 Figure 2.2: UART harness address map



UART harness address map

Source: 02_rs_uart_address_map.dot

5.2 Window summary

Base	Size	Window	What answers it
0x0000_0000	4 KB	rs_loop_apb	The harness CSR block rs_loop_regs, reached through the shared apb4_to_peakrdl shim. This is where BUILD_ID, SCRATCH, PROFILE, TOPOLOGY, control, status, verdict tallies and bandwidth meters live.
0x0001_0000	4 KB	rs_regs_apb	**As built today:** on the AXI4 flavour the pipeline's axi4_intf_master_observer answers this window; on the AXIS flavour it is tied to PRDATA=0, PREADY=1 so a stray access cannot hang the host bus. **PLANNED:** the codec's own rs_regs block from PRD D9

Base	Size	Window	What answers it
			(standalone codec ID, PROFILE, CTRL, job fields, statistics, interrupts).
`0x0002_0000`	4 KB	`obs_apb`	Always live. An `axis4_intf_observer` watches the four AXIS seams in codeword order: message-in, codeword-out-of-encoder, codeword-into-decoder, message-out. On the AXI4 flavour the seams are tied off and the observer reads zero, but it still responds.

The fabric definition is frozen in `rtl/bridges/generated/bridge_rs_loop_axil/bridge_rs_loop_axil.toml`. `bin/regen_bridges.sh` regenerates the RTL if the windows move.

5.3 rs_loop_apb register offsets

The register block is generated from `build-loop/rtl/rs_loop_regs.rdl`. The host accesses every register **by name** through `UartRegisterMap` over the generated `build-loop/dv/tbclasses/rs_loop_regs_regmap.py`; no Python code hard-codes an offset.

Offset	Register(s)	Purpose
0x000	BUILD_ID	Identity: ASCII `RSLP` (board profile) or `RSLS` (small Nexys A7 profile).
0x004	SCRATCH	Read-write sanity register.
0x020	PROFILE	`{n[15:0], t[23:16], m[27:24], spb[31:28]}`.
0x084	TOPOLOGY	What the bitstream built: decoders, solver, comparator, datapath, erasure support.
0x008	CTRL	`soft_reset`, `clear`, `bypass`, `throttle_a`, `throttle_b`, `inj_seed_on_start`.
0x00C	GEN_BLOCKS	Blocks per run.
0x010	GEN_SEED	Pattern generator LFSR seed.
0x014	INJ_CFG	Injector mode, erasure `mark`, error count, rate.
0x018	INJ_SEED	Injector LFSR seed.
0x088	INJ_CNT	Cluster count range (mode 4).
0x08C	INJ_LEN	Cluster length range (mode 4).
0x080	GO	Single-pulse kick; every consumer starts

Offset	Register(s)	Purpose
		off the same cycle.
0x01C	STATUS	busy, done flags, error flags, AXI4 stage/overflow.
0x024–0x02C	CRC_EXPECTED, CRC_A, CRC_B	Generator and checker CRC-32s.
0x030–0x034	PKTS_A, PKTS_B	Blocks received by each checker.
0x038	CYCLES	Cycles from start until both checkers have every block.
0x03C–0x060	BLK_OK_A/B, BLK_CORR_A/B, BLK_UNC_A/B, BLK_FRAME_A/B, SYM_CORR_A/B	Per-decoder verdict tallies.
0x064–0x070	INJ_SYMBOLS, INJ_BLOCKS, INJ_OVER_T, INJ_LAST	Injector statistics.
0x074–0x07C	CMP_DATA_MISMATCH, CMP_STATUS_MISMATCH, CMP_BEATS	A-vs-B comparator (simulation / ENABLE_COMPARE=1 only).
0x090–0x0CC	OBS_IN_*, OBS_OUT_*, OBS_CW_OUT_*, OBS_CW_IN_*	Four seams, four buckets each (productive, backpressure, starvation, idle).

The complete generated register documentation is at `build-loop/rtl/generated/rs_loop_regs/docs/rs_loop_regs.md`.

5.4 By-name access

The host driver (`build-loop/host/rs_loop.py`) opens three `UartRegisterMap` instances:

- `regs` at `HARNESS_BASE = 0x0` using `rs_loop_regs_regmap.py`.
- `obs_axi4` at `OBS_AXI4_BASE = 0x0001_0000` using `obs_regs_top_regmap.py`.
- `obs_axis` at `OBS_AXIS_BASE = 0x0002_0000` using `obs_regs_top_regmap.py`.

Because the bridge is injectable, the same driver runs over pyserial on the board and over the cocotb UART in simulation.

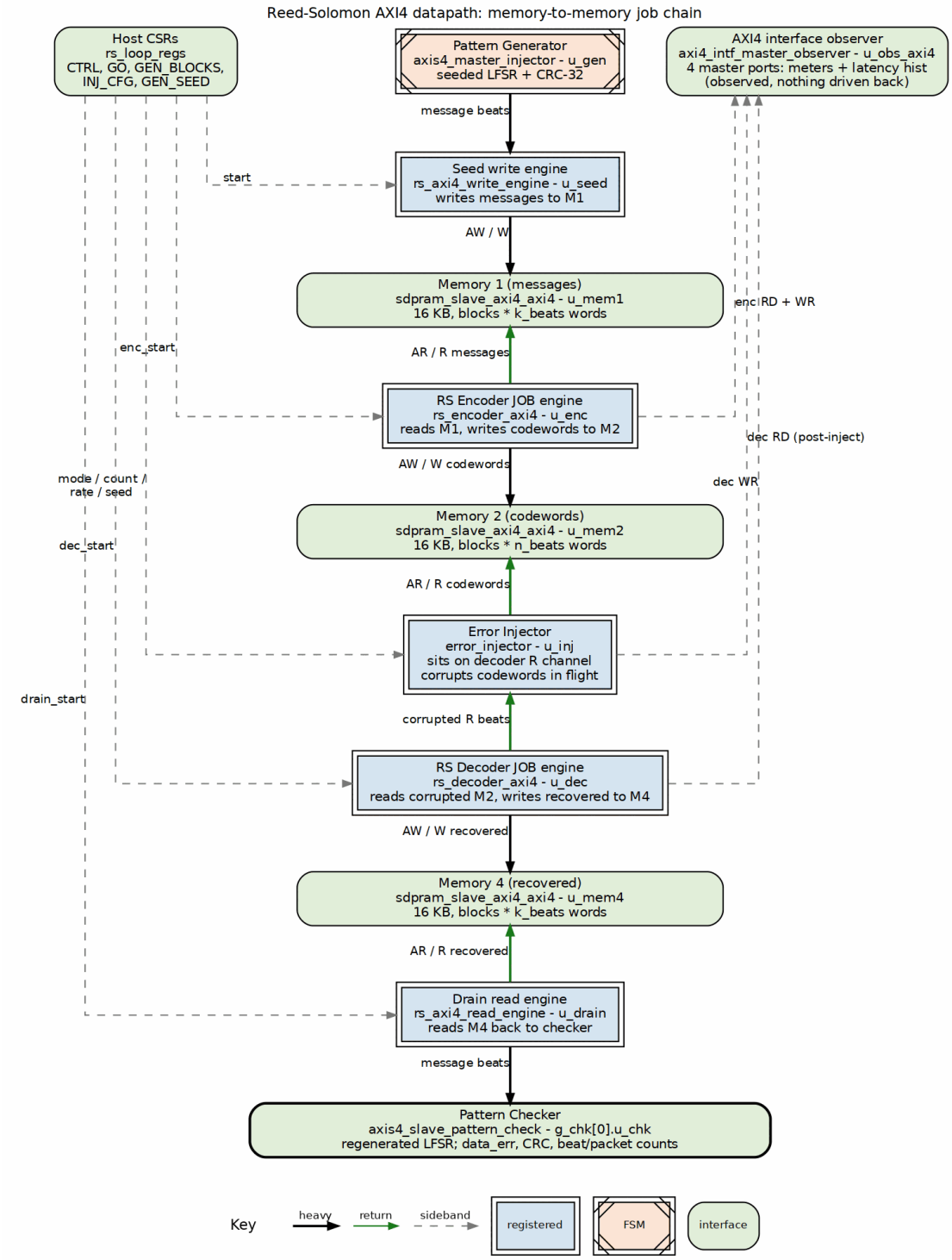
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 AXI4 Memory-to-Memory Datapath

The default board-validated flavour is the AXIS stream pipe shown in How The Pieces Connect. The same harness also builds an AXI4 flavour (`IFACE="AXI4"`) that replaces the middle of the pipe with a memory-to-memory job chain. The generator, checkers, CSRs and verdict logic are unchanged, so a run reports itself identically whichever middle is built.

6.1 Figure 2.3: AXI4 datapath



Reed-Solomon AXI4 datapath

Source: 03_rs_axi4_pipeline.dot

6.2 Job chain

The chain is four sequential jobs over three dual-port memories:

1. **Seed write** — the generator's message stream is written to **M1**.
2. **Encode** — the encoder reads **M1** and writes codewords to **M2**.
3. **Decode** — the decoder reads **M2** and writes recovered messages to **M4**. The injector sits on the decoder's read channel, so the codewords are corrupted in flight; no extra memory hop is needed.
4. **Drain read** — the drain engine reads **M4** and feeds the pattern checker.

Stage sequencing is done by rising-edge detectors on each stage's `cfg_done`, not by a state machine. Each memory has exactly one writer and one reader, so there is no arbitration on the AXI4 channels.

6.3 Memories

All three memories are `sdpram_slave_axi4_axi4` instances with `MEM_DEPTH = 4096` 32-bit words (16 KB each). A single AXI4 run cannot exceed what one memory holds; `STATUS.axi4_overflow` refuses the kick rather than clamping or silently wrapping.

6.4 Observer

The AXI4 flavour's `rs_regs_apb` window (`0x0001_0000`) hosts an `axi4_intf_master_observer` instantiated inside `rs_axi4_pipeline`. It watches the codec's four master ports:

Port	Direction	What it carries
encoder RD	read	M1 message read
encoder WR	write	M2 codeword write
decoder RD	read	M2 codeword read, after the injector
decoder WR	write	M4 recovered write

The observer provides the same bus-meter cycle buckets as the AXIS observer, plus log2 latency histograms. `ENABLE_MON_TAPS = 0` keeps it purely passive so it cannot throttle the datapath.

6.5 Verdict path

Because the decoder reaches its verdict during the decode stage while the beats arrive at the checker later, during the drain stage, the per-block tally cannot follow the AXI4 data path. The pipeline accumulates its own verdict totals (`blk_ok`, `blk_corr`, `blk_unc`, `blk_frame`, `sym_corr`) and the CSR block selects those sources when `TOPOLOGY.iface` indicates AXI4.

7 The FPGA-Testing Choices, And Why

7.1 The dual-solver loop as mutual oracle

The AXIS datapath feeds the same corrupted blocks to two `rs_decoder_core` instances, one with `KES_ALGO=RIBM` and one with `KES_ALGO=EUCLID`. Each decoder's output is compared beat-by-beat against the generator's regenerated LFSR pattern in its own `axis4_slave_pattern_check`, so a correction is judged by a reference that never saw the errors — not by the other decoder. A separate comparator then requires the two decoders to agree on every beat and on every block's verdict. That makes a solver bug visible as a disagreement even when both decoders "correct." On the board images this comparator is present only in simulation (`ENABLE_COMPARE=1`); the production board matrix carries one solver per bitstream to save area and keep the handshake clean for the bandwidth meters. The cross-solver check then happens by running the identical deterministic campaign on the riBM and Euclid images and requiring matching numbers.

7.2 Determinism makes the riBM and Euclid images cross-oracles

The host-side RNG seed defaults to 1. With that default, the random, soak, clusters, localized, and badblock campaigns are fully deterministic: the riBM and Euclid images run bit-identical workloads. On the 2026-10-05 Genesys 2 battery all four single-decoder images passed 7/7 campaigns, and the matching verdict counts between riBM and Euclid are themselves the cross-check. You do not need the on-chip comparator to know the two solvers agree — the campaigns prove it.

7.3 Injector modes map to real failure mechanisms

The shared `error_injector` operates on 8-bit symbols, because that is RS's natural unit. Mode 1 exact-count places exactly e errors per block at uniformly random distinct positions, which is what makes the e = t / e = t + 1 boundary sharp. Mode 2 burst and mode 3 rate model raw channel bit-error behavior. Modes 4 clusters and 5 localized model spatially correlated faults — a bad DRAM row, a disturbed NAND page, or a failed symbol column. Mode 6 badblock models the tail of NAND retention: most blocks clean, a hard minority very bad. Mode 7 debug is the deterministic bring-up walk. When `INJ_CFG.mark` is set, the injector's hit mask rides the decoder `in_erasure` sideband, turning any of these placement modes into an erasure run where the decoder is told which symbols were corrupted.

7.4 Post-encoder injection protects parity symbols

Errors are injected after the encoder, not in the generator. An error in the generator's data would be encoded faithfully and would be invisible to the code. Placing the injector on the coded stream means parity symbols are corrupted too, which is the workload the decoder actually has to handle. That is why the generator has no error-injection mode and the injector is a separate block.

7.5 UART over a debug core, with a lock/readback safety pair

The harness uses a plain UART instead of a proprietary debug core, so the host side is just pyserial and Python scripts. Board safety is a pair: `board_lock.sh` serializes programming flows by the board's JTAG serial, and `fpga_board.py` reads the JTAG chain through `jtag_readback.tcl` before programming to confirm the expected Genesys 2 serial (`200300B818A0`) is present with a device behind it. A lock prevents collisions; a readback detects misattribution. When the hw_server deviceless-enumeration race refused about half of programming attempts, the readback path was changed to bounce hw_server and retry, bounded at five. The verdict is recorded beside the bitstream sha256 so a result file cannot be mistaken for verified if the identity check was inconclusive.

7.6 Genesys 2 over Nexys A7

The Kintex-7 XC7K325T-2 was chosen because it has room for the full harness plus observers and still closes timing with positive slack. The four Genesys 2 images routed with WNS from +0.875 ns to +1.822 ns. The Nexys A7-100T flow is kept as a target option (`RS_TARGET=nexys_a7_100t`) and the original image names and report layout stay byte-identical, but the Genesys 2 is the primary target for harness-class work.

7.7 AXIS and AXI4 flavors cover both integration styles

The AXIS flavor is a single streaming pipe: generator, encoder, injector, decoder, checker. The AXI4 flavor replaces the middle with `rs_axi4_pipeline`, a memory-to-memory job chain over block RAM. Both use the same generator, checker, CSRs, and verdict logic, so a run reports itself identically whichever middle is built. The AXI4 flavor refuses a `GEN_BLOCKS` value that would exceed the job memories — it declines rather than clamps, because clamping would answer a question the host did not ask. Running the same deterministic campaign on both flavors is a free sanity check: at e = 8 the AXI4 flavor takes 381.0 cycles/block against 199.2 on AXIS. The two numbers must be consistent for the same workload, and they are. The million-block soak ran on `axi4_ribm`; an earlier plan to prefer AXIS for soaks proved unnecessary.

8 Operating It

To run on the Genesys 2:

```
cd projects/fpga-systems/Genesys2/ecc-ip/reed-solomon
bin/run_smoke.py --board genesys2 --port /dev/ttyUSB0 --sequences init
smoke sweep
```

For the Nexys A7 small profile, build the bitstream with `make -C build-loop bitstream RS_TARGET=nexys_a7_100t RS_PROFILE=small` and drive the same sequences over the A7's UART; `init` proves the "RSLS" BUILD_ID and the PROFILE CSR reports n=64, t=4, so the host programs pick up the geometry from the board.

See `README.md` and `Makefile` for the full target list (bitstream, lint, sim, regmap) and `build-loop/host/host_rs_loop.py` for the lower-level CLI. Build evidence and the board-validation record live in `stable/MANIFEST.md` and the four `stable/reports/genesys2_*` directories. Campaign transcripts turn into reports with `projects/fpga-systems/bin/report_battery.py` (JSON artifacts, correction-boundary / decode-cost / soak-timeline figures, and a generated `FINDINGS.md`); published runs live under `stable/results/`. The run evidence — the 2026-10-05 four-image battery and the one-million-block soak — is rolled up in the companion *Reed-Solomon Board Validation Report* (`docs/Reed_Solomon_Board_Validation_v0.1.pdf`). The math behind the codec is in the component HAS chapter 7 at `projects/components/ecc-ip/reed-solomon/docs/reed_solomon_has/ch07_understanding_the_math/`. The sister BCH harness has the matching guide at `../../bch/docs/Binary_BCH_UART_Harness_v0.1.pdf`.

9 What Is Deliberately Not Board-Tested

A few things are intentionally left in simulation or are still open:

- The 2026-10-05 board battery ran the four single-decoder images. The dual-decoder comparator with `ENABLE_COMPARE=1` is the sim/DV domain; it caught real harness bugs (dropped/duplicated beats under skewed drains) without costing a bitstream.
- The erasure `f = 2t` boundary is a known open bug, filed as `vault/tracking/projects/components/ecc-ip/reed-solomon/task/open/TASK-005.md`. `f = t` corrects and `f = 2t + 1` refuses on the board; `f = 2t` is flagged uncorrectable even though the design intent says it should correct.
- The component DV matrix — 195 cells per `TASK-001.md` — exercises the codec in ways the board harness does not replicate.
- The board validates two profiles: RS(252,236) t=8, S=4 on the Genesys 2, and the small RS(64,56) t=4 on the Nexys A7 (`RS_PROFILE=small`, issue #83); profiles outside those two are covered in simulation and component DV.
- The 2026-10-05 million-block soak is statistical, not exhaustive; its counters, the 2-of-74,240 mis-decodes, and the bounded-rate pass criterion are reported in the Board Validation Report.